

Algorithm Lab (CS2271)

Assignment 8

Department of Computer Science and Technology
Indian Institute of Engineering Science and Technology, Shibpur

Q1. Fibonacci Series

In a smart agriculture system, many sensors are placed in fields to collect data such as soil moisture, temperature, and humidity. These sensors send the collected data to a central server through a wireless network. Sometimes, due to weak network connectivity, data packets fail to reach the server successfully. To handle this situation, the system waits for some time before trying to send the data again. Instead of waiting for the same time every time, the system increases the waiting time gradually to reduce network congestion. One simple method used in many computer networks is the **Fibonacci backoff strategy**, where the waiting time follows the Fibonacci sequence. For example, if the first retry fails, the system waits **1 second**, then **1 second**, then **2 seconds**, then **3 seconds**, then **5 seconds**, **8 seconds**, and so on. However, when the number of retries becomes large, calculating Fibonacci numbers using the simple recursive method becomes slow because the same values are calculated many times. Therefore, developers want to compare different methods to calculate Fibonacci numbers efficiently. Write a **C program** to compute the **n-th Fibonacci number** using the following two approaches:

1. Recursive Method

Implement the traditional recursive Fibonacci algorithm where each value is calculated using repeated function calls.

2. Dynamic Programming Method

Implement the Fibonacci algorithm using **Dynamic Programming (Memoization or Tabulation)** where previously calculated values are stored and reused.

After implementing both methods, perform a comparative analysis based on the following:

- Time Complexity in the best case, average case, and worst case.
- Space Complexity in the best case, average case, and worst case.
- Execution performance when calculating large Fibonacci numbers.

Q2. Matrix Chain Multiplication

In many cloud computing and machine learning systems, large numbers of matrices are multiplied together to perform computations such as data transformations, neural network training, and image processing. Suppose several matrices need to be multiplied in sequence:

$$A_1, A_2, A_3, \dots, A_n$$

Even though the final multiplication result remains the same, the number of calculations required can be very different depending on the order in which the matrices are multiplied. For example, suppose we have three matrices with dimensions:

- (10×30)
- (30×5)
- (5×60)

If we change the order of multiplication, the total number of scalar multiplications required will also change. Some orders require fewer calculations and therefore execute faster. To improve performance, the system must determine the **best order of multiplying matrices that minimizes the total number of scalar multiplications**. Write a **C program** to determine the **minimum number of scalar multiplications** required to multiply a sequence of matrices using the following approaches:

1. Recursive Approach

Implement a recursive algorithm that checks all possible ways of multiplying the matrices and finds the minimum cost.

2. Dynamic Programming Approach (Matrix Chain Multiplication)

Implement the dynamic programming solution that stores intermediate results in a table and avoids repeated calculations.

After implementing both methods, perform a comparative analysis based on the following factors:

- Time Complexity in the best case, average case, and worst case.
- Space Complexity in the best case, average case, and worst case.
- Performance when the number of matrices increases.

Q3. 0/1 Knapsack Problem

A delivery and logistics company transports packages using trucks that have a **maximum weight capacity**. The company must decide which packages should be loaded into a truck so that the **total profit from delivered packages is as high as possible**, without exceeding the weight limit of the truck. Each package has two properties:

- **Weight** – how much space or load it occupies in the truck
- **Profit (Value)** – the benefit gained from delivering that package

Since each package can either be **selected or not selected**, this problem is known as the **0/1 Knapsack Problem**. When the number of packages becomes large, checking all possible combinations of packages becomes very slow. Therefore, more efficient algorithms are needed.

Write a **C program** to solve the **0/1 Knapsack Problem** using the following approaches:

1. **Recursive / Brute Force Approach**

Implement a recursive solution that checks all possible combinations of packages to find the maximum profit.

2. **Dynamic Programming Approach** Implement the dynamic programming solution using a table where intermediate results are stored to avoid repeated calculations.

After implementing both methods, perform a comparative analysis based on:

- Time Complexity in the best case, average case, and worst case.
- Space Complexity in the best case, average case, and worst case.
- Efficiency when the number of packages becomes very large.